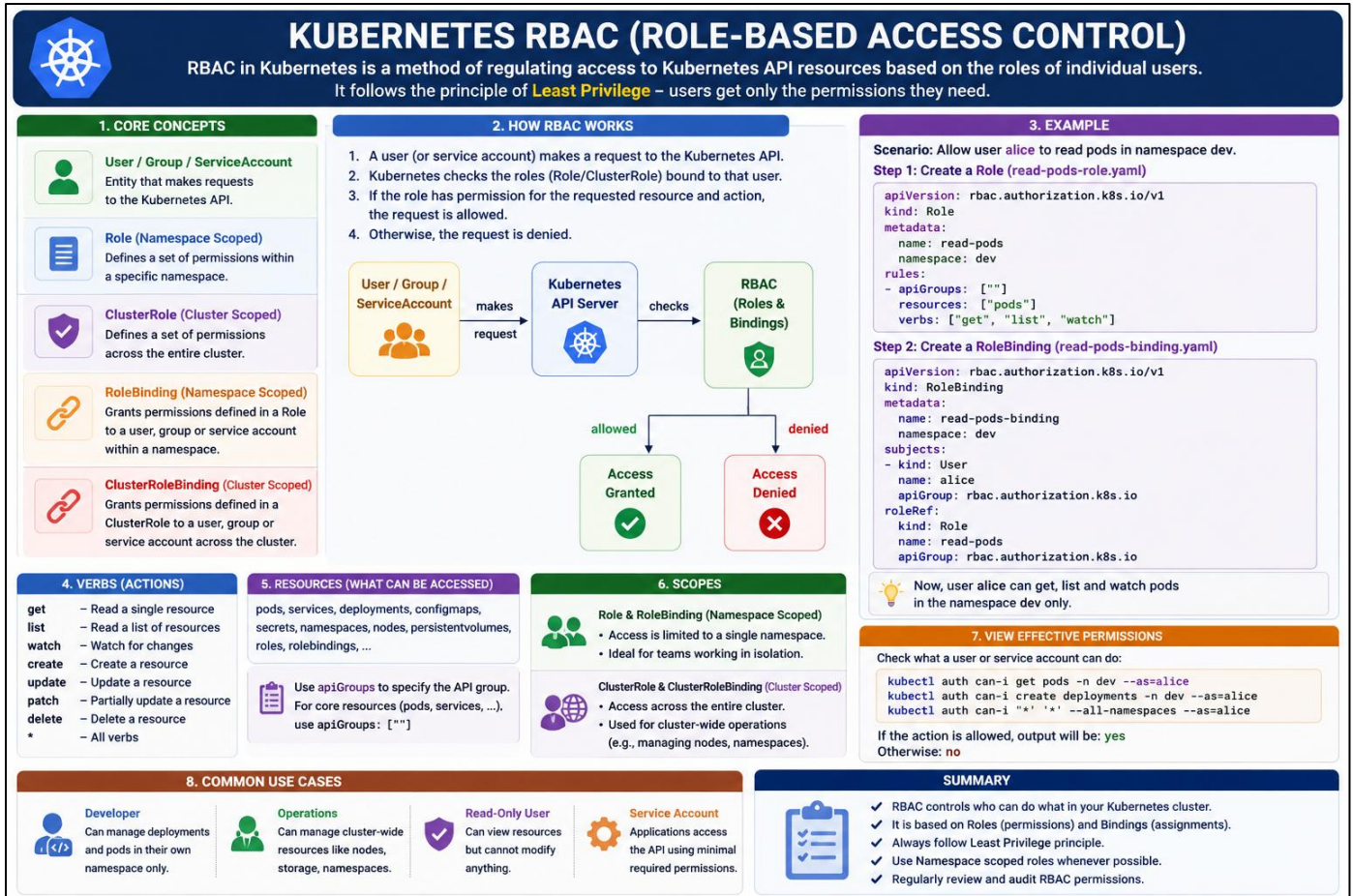


K8s RBAC Service Account Concept





RBAC in Kubernetes

RBAC answers one question:

"Who can do what in Kubernetes?"



1. WITHOUT RBAC

Suppose there are 3 team members:



Bhavana
(DevOps)



Ravi
(Developer)



Sita
(Tester)

Without RBAC, everyone can:

- Delete Pods ❌
- Delete Deployments ❌
- Access Secrets ❌
- Modify Production Resources ❌



This is risky.

2. WITH RBAC – We define permissions



DEVELOPER

Can:

- ✓ View Pods
- ✓ Create Deployments

Cannot:

- ✗ Delete Namespace
- ✗ Access Secrets



TESTER

Can:

- ✓ View Pods

Cannot:

- ✗ Create Deployments
- ✗ Delete Pods



DEVOPS ENGINEER

Can:

- ✓ Manage everything



3. REAL-TIME EXAMPLE

Suppose you have a namespace:

production



Ravi
(Developer)



Namespace:
production

Developer Ravi should only view pods.

Create a Role:

```
rules:  
- apiGroups: [""]  
  resources: ["pods"]  
  verbs: ["get", "list", "watch"]
```

This means:

Ravi can:

```
kubectl get pods  
kubectl describe pod
```

Ravi cannot:

```
kubectl delete pod  
kubectl create deployment
```

4. HOW RBAC WORKS



**USER /
SERVICE ACCOUNT**
(Identity)



**ROLE /
CLUSTERROLE**
(Permissions)



**ROLEBINDING /
CLUSTERROLEBINDING**
(Assignment)



**KUBERNETES
API SERVER**
(Enforcement)

PRINCIPLE OF LEAST PRIVILEGE



Give users only the minimum permissions they need to do their job – nothing more.

5. EASY INTERVIEW ANSWER

“RBAC (Role-Based Access Control) is a Kubernetes security mechanism used to control who can access Kubernetes resources and what actions they can perform. It works using Roles/ClusterRoles (permissions) and RoleBindings/ClusterRoleBindings (assignments). RBAC follows the principle of least privilege, giving users only the permissions they need.”

6. WHY RBAC IS USED IN COMPANIES



To prevent unauthorized users from modifying or deleting critical Kubernetes resources, especially in production environments.

- Protects production workloads
- Prevents human errors
- Enforces security & compliance
- Gives the right access to the right person

7. QUICK SUMMARY

Component	What it is	Scope	Used for
Role	Define permissions within a namespace	Namespace	What actions can be done on which resources
RoleBinding	Assign a Role to a User/Group/ServiceAccount	Namespace	Give the defined permissions to an identity
ClusterRole	Define permissions across the cluster	Cluster-wide	Access cluster-wide resources (nodes, namespaces, etc.)
ClusterRoleBinding	Assign a ClusterRole to a User/Group/ServiceAccount	Cluster-wide	Give cluster-wide permissions to an identity



MEMORY TRICK

Role
= Permissions (one namespace)
RoleBinding
= Assign Permissions (one namespace)
ClusterRole
= Permissions (entire cluster)
ClusterRoleBinding
= Assign Permissions (entire cluster)



IDENTITY PROVIDERS (IDP) IN KUBERNETES

IDP = Login System for Kubernetes



WHAT IS AN IDP?

An Identity Provider (IDP) is a system that verifies a user's identity and tells Kubernetes who the user is.



IDP =
Login System
for Kubernetes

REAL-LIFE EXAMPLE



Gmail using your Google account



GitHub using your Microsoft account



Google and Microsoft are acting as Identity Providers.
Similarly, Kubernetes can trust an IDP to authenticate users.

WITHOUT AN IDP



User

kubectl command



Kubernetes API Server



Kubernetes doesn't know who the user is.

WITH AN IDP



User

Login



Identity Provider (IDP)



Authentication Token



Kubernetes API Server



RBAC Check



Access Granted / Denied

Flow:

- 1 User logs in to IDP
- 2 IDP authenticates the user
- 3 IDP sends token to Kubernetes
- 4 Kubernetes performs RBAC check
- 5 Access is granted or denied

COMMON IDENTITY PROVIDERS (IDPs)



AWS IAM
(for EKS)



Azure AD
(for AKS)



LDAP

okta

Okta



SSO
(Single Sign-On)



Keycloak
(Open Source
Identity Broker)

INTERVIEW ANSWER



An Identity Provider (IDP) in Kubernetes is an external authentication system such as Azure AD, Okta, LDAP, Active Directory, Google Identity, or Keycloak. It verifies user identities and provides authentication information to the Kubernetes API server. After authentication, Kubernetes RBAC determines what actions the user is authorized to perform.



KEY TAKEAWAY



User tries to
access Kubernetes



IDP authenticates
the user



IDP sends
authentication
token



Kubernetes API
Server validates
token



RBAC checks
permissions



Access
Granted / Denied



SERVICE ACCOUNT IN KUBERNETES

An identity used by applications (pods) to communicate with the Kubernetes API Server.



service account

In simple words:



User Account → Used by Humans



Service Account → Used by Pods/Applications

REAL-LIFE EXAMPLE

Human User

You run:

```
kubectl get pods
```

Kubernetes identifies:



User = Bhavana

using:

IDP + RBAC

Pod/Application

Suppose a pod wants to:

- Read ConfigMaps
- Read Secrets
- Create Pods
- Access Kubernetes API

Kubernetes identifies:



ServiceAccount = app-sa

instead of a human user.

EXAMPLE ARCHITECTURE

Human Access



Bhavana

RBAC



Kubernetes API

Application Access



Application Pod

Service Account
app-sa



Kubernetes API

WHY DO WE NEED SERVICE ACCOUNTS?

Imagine a monitoring application:



Prometheus Pod

It needs to:

- ✓ Read Pods
- ✓ Read Nodes
- ✓ Read Services



How will Kubernetes know who it is?
Using a **Service Account**.

REAL-TIME EXAMPLES



Argo CD

Pods need permissions to:

- ✓ Create Deployments
- ✓ Update Services
- ✓ Create Pods



Argo CD uses Service Accounts.



Prometheus

Prometheus

Needs permissions to:

- ✓ Read Pods
- ✓ Read Nodes
- ✓ Read Metrics



Uses Service Accounts.



Jenkins in Kubernetes

Needs permissions to:

- ✓ Create Pods
- ✓ Delete Pods



Uses Service Accounts.

HOW IT WORKS

Pod / Application

Service Account
app-sa

RBAC
(Role / RoleBinding)



Kubernetes API Server

- 1 Pod uses the Service Account (app-sa).
- 2 The Service Account is bound to a Role with permissions.
- 3 Kubernetes API Server checks the permissions.
- 4 Access is granted or denied based on RBAC rules.

KEY BENEFITS

- ✓ Gives applications a secure identity to access the Kubernetes API. 
- ✓ Allows fine-grained permissions using RBAC. 
- ✓ Follows the principle of least privilege (only required permissions). 
- ✓ Improves security and control for applications running in the cluster. 

INTERVIEW ANSWER



A Service Account is a Kubernetes identity used by applications and pods to authenticate with the Kubernetes API Server. Unlike user accounts, which are used by humans, service accounts are used by workloads running inside the cluster. Service accounts are commonly combined with RBAC roles and role bindings to grant pods only the permissions they need.



Pod / Application



Service Account



Kubernetes API Server



What is a Role in Kubernetes?



A Role is a set of permissions that defines:
What actions can be performed on which resources inside a namespace?



Think of a Role as a **job description**.

REAL-LIFE EXAMPLE

Suppose you're a Tester.

You should be able to:

- ✓ View Pods
- ✓ View Services

You should NOT be able to:

- ✗ Delete Pods
- ✗ Create Deployments



TESTER



These permissions are stored in a **Role**.

EXAMPLE ROLE

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: lab
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```



WHAT DOES THIS MEAN?

resources: ["pods"]



Access only to Pods.

verbs: ["get", "list", "watch"]

Allowed actions:



kubectl get pods

Get the list of pods



kubectl describe pod nginx

Show details of a pod



kubectl get pods -w

Watch pods in real time

NOT ALLOWED



kubectl delete pod nginx

Delete a pod



kubectl create deployment app1

Create a deployment

KEY TAKEAWAYS



A Role defines permissions within a specific **namespace**.



It specifies which **resources** can be accessed and what actions (**verbs**) are allowed on them.



A Role alone does nothing; it must be assigned using a **RoleBinding**.



Roles help follow the principle of **least privilege** by giving only the necessary permissions.



In short:

Role = Permissions (What can be done on which resources in a namespace)

RoleBinding = Assignment of those permissions to a user or service account





What is a RoleBinding in Kubernetes?

A RoleBinding connects a Role to a User, Group, or Service Account.



In simple words:



Role = What permissions are available



RoleBinding = Who gets those permissions

REAL-LIFE EXAMPLE

Think of a company.



Role: Tester Role

Permissions:

- ✓ View Pods
- ✓ View Services
- ✗ Delete Pods
- ✗ Create Deployments



The Role only defines permissions. Nobody has those permissions yet.

ROLEBINDING

Now we assign the Role to someone:

Tester Role

RoleBinding



Now Bhavana gets those permissions.

SIMPLE EXAMPLE (READ PODS ONLY)



ServiceAccount
dev-user
(dev namespace)



RoleBinding
read-pods-binding



Role
pod-reader:
Permissions:
• get pods
• list pods
• watch pods

KUBERNETES EXAMPLE (STEP BY STEP)

1 Step 1: Create Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role

metadata:
  name: pod-reader
  namespace: lab

rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```



This Role says:
Can view Pods

2 Step 2: Create Service Account

```
apiVersion: v1
kind: ServiceAccount

metadata:
  name: tester-sa
  namespace: lab
```



tester-sa

3 Step 3: Create RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding

metadata:
  name: pod-reader-binding
  namespace: lab

subjects:
- kind: ServiceAccount
  name: tester-sa
  namespace: lab

roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```



HOW IT WORKS



tester-sa

RoleBinding

pod-reader Role



Read Pods

Now any Pod using:

```
serviceAccountName: tester-sa
```

can:

```
> kubectl get pods
```

but cannot:

```
> kubectl delete pod nginx
```

KEY POINTS



Role defines the permissions (what can be done).



RoleBinding assigns those permissions to a User, Group, or Service Account.



RoleBinding is always created in the same namespace as the Role.



It acts as a link between identity and Role.

EASY ANALOGY



Role
= Access Card



RoleBinding
= Giving the card to a person



User/ServiceAccount
= Person



Result
= Gets permissions

Example:

Access Card: Can enter Server Room

RoleBinding: Give card to Bhavana

Result: Bhavana can enter Server Room



INTERVIEW ANSWER



A RoleBinding is an RBAC object that grants the permissions defined in a Role to a User, Group, or Service Account within a namespace. It acts as a link between an identity and a Role.

In short:

Role = Permissions

RoleBinding = Assignment of Permissions



Remember:

Role tells Kubernetes **WHAT** can be done. RoleBinding tells Kubernetes **WHO** can do it.



What is a ClusterRole in Kubernetes?



A ClusterRole is similar to a Role, but it works at the **cluster level** instead of a single namespace.

In simple words:



Role = Permissions inside one namespace



ClusterRole = Permissions across the entire cluster

ROLE VS CLUSTERROLE

ROLE

Suppose you create a Role in namespace lab:

✓ Can view Pods

Only in namespace: **lab**

It **cannot** access pods in:

- ✗ default
- ✗ prod
- ✗ dev

VS

CLUSTERROLE

A ClusterRole can access resources across all namespaces.

- ✓ Can view Pods
- ✓ Can view Nodes
- ✓ Can view Namespaces



Across the entire cluster

WHY DO WE NEED CLUSTERROLE?

Some resources are cluster-wide.

Examples:



Nodes



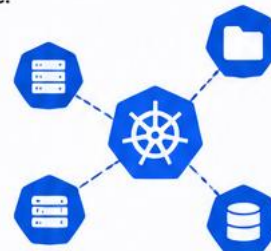
Namespaces



StorageClasses



PersistentVolumes



These resources don't belong to a single namespace.

A normal Role cannot manage them.

EXAMPLES

EXAMPLE 1: READ NODES

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]
```



This allows:

>.. `kubectl get nodes`

>.. `kubectl describe node`

EXAMPLE 2: READ PODS IN ALL NAMESPACES

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-reader-all
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```



Now the user can run:

>.. `kubectl get pods -A`

and see pods from:

default

kube-system

lab

prod

ROLE VS CLUSTERROLE

Feature	Role	ClusterRole
Scope	Namespace scoped	Cluster scoped
Access	Access resources in one namespace	Access resources across all namespaces
Binding	Bound using RoleBinding	Usually bound using ClusterRoleBinding
Cluster-wide Resources	Cannot manage Nodes, Namespaces, etc.	Can manage Nodes, Namespaces, StorageClasses, etc.

INTERVIEW ANSWER



A ClusterRole is an RBAC object that defines permissions at the **cluster level**.

Unlike a Role, which is limited to a namespace, a ClusterRole can grant access to cluster-wide resources such as nodes, namespaces, storage classes, and resources across multiple namespaces.

ClusterRoles are assigned using ClusterRoleBindings.

KEY TAKEAWAY



Role
Works in one namespace only



ClusterRole
Works across the entire cluster



Assigned using (Cluster)RoleBinding to Users, Groups or ServiceAccounts



Controls who can do what in Kubernetes at namespace or cluster level



In short:

Role = Permissions in one namespace

ClusterRole = Permissions across the entire cluster



What is a ClusterRoleBinding in Kubernetes?

A ClusterRoleBinding connects a ClusterRole to a User, Group, or ServiceAccount.



ClusterRole = What permissions are available across the cluster

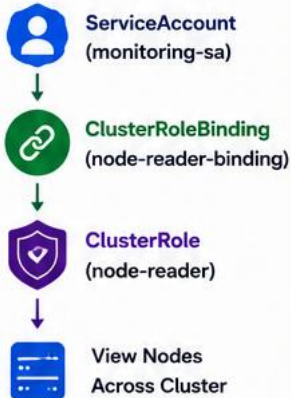


ClusterRoleBinding = Who gets those permissions



ClusterRole Alone Does Nothing
Just like a Role, a ClusterRole must be assigned.
You use: **ClusterRoleBinding**

EXAMPLE ARCHITECTURE



EXAMPLE (STEP BY STEP)

1 Step 1: Create a ClusterRole

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
```

This ClusterRole allows:

- ✓ kubectl get nodes
- ✓ kubectl describe nodes

2 Step 2: Create a ServiceAccount

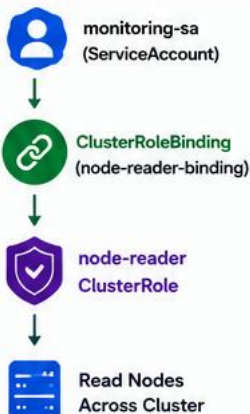
```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitoring-sa
  namespace: lab
```

ServiceAccount
monitoring-sa
(namespace: lab)

3 Step 3: Create ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-reader-binding
subjects:
- kind: ServiceAccount
  name: monitoring-sa
  namespace: lab
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

HOW IT WORKS



Now any pod using:

```
serviceAccountName: monitoring-sa
```

can access:

```
> kubectl get nodes
```

across the entire cluster.
(Any namespace)



Entire Kubernetes Cluster

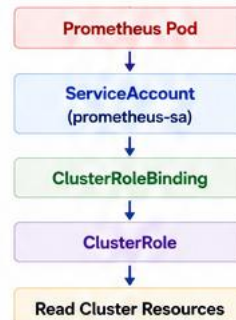
REAL-TIME EXAMPLES

PROMETHEUS



Prometheus needs to discover:

- ✓ Nodes
 - ✓ Pods
 - ✓ Services
 - ✓ Endpoints
- in all namespaces.



ARGO CD



Argo CD needs permissions to:

- ✓ Create Deployments
 - ✓ Update Services
 - ✓ Read Namespaces
- across the cluster.

Uses:



ROLEBINDING vs CLUSTERROLEBINDING

Feature	RoleBinding	ClusterRoleBinding
Scope	Namespace scope	Cluster scope
Grants	Grants Role permissions	Grants ClusterRole permissions
Access	Access limited to a namespace	Access across the cluster
Purpose	Used for local access	Used for global access

INTERVIEW ANSWER



A **ClusterRoleBinding** is an RBAC object that grants the permissions defined in a **ClusterRole** to a User, Group, or ServiceAccount across the entire Kubernetes cluster.



It is used when permissions are needed beyond a single namespace, such as accessing nodes, namespaces, or resources in multiple namespaces.

KEY TAKEAWAYS



ClusterRole defines cluster-wide permissions.



ClusterRoleBinding assigns those permissions to an identity.



Works at the cluster scope (not limited to any namespace).



Commonly used by monitoring tools, GitOps tools, and cluster admins.



Provides secure access to cluster resources across all namespaces.

Practical:

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
$ eksctl create cluster --name clusterrbac --nodegroup-name b17dng --node-type m7i-flex.large --nodes 3 --managed
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ eksctl get cluster
NAME          REGION    EKSCtl  CREATED
clusterrbac   us-east-1  True
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-192-168-30-227.ec2.internal     Ready    <none>   3m19s  v1.34.9-eks-93b80c6
ip-192-168-63-25.ec2.internal      Ready    <none>   3m19s  v1.34.9-eks-93b80c6
ip-192-168-8-123.ec2.internal      Ready    <none>   3m19s  v1.34.9-eks-93b80c6
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ aws sts get-caller-identity
{
  "UserId": "459532536509",
  "Account": "459532536509",
  "Arn": "arn:aws:iam::459532536509:root"
}
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ kubectl api-resources | grep pod
pods                po                v1
true                Pod
podtemplates        v1
true                PodTemplate
horizontalpodautoscalers hpa              autoscaling/v2
true                HorizontalPodAutoscaler
pods                metrics.k8s.io/v1beta1
true                PodMetrics
poddisruptionbudgets pdb              policy/v1
true                PodDisruptionBudget
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ kubectl api-resources -o wide | grep pod
pods                po                v1
true                Pod
create,delete,deletecollection,get,list,patch,update,watch all
podtemplates        v1
true                PodTemplate
create,delete,deletecollection,get,list,patch,update,watch
horizontalpodautoscalers hpa              autoscaling/v2
true                HorizontalPodAutoscaler
create,delete,deletecollection,get,list,patch,update,watch all
pods                metrics.k8s.io/v1beta1
true                PodMetrics        get,list
poddisruptionbudgets pdb              policy/v1
true                PodDisruptionBudget
create,delete,deletecollection,get,list,patch,update,watch
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ kubectl get sa
NAME          SECRETS  AGE
```


default 0 12m

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
$ cd practice/19_June-RBAC-ServiceAccount/
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ ll
total 3542
-rw-r--r-- 1 Bhavana 197121 89 Jun 22 19:00 1-sa-dev.yaml
-rw-r--r-- 1 Bhavana 197121 274 Jun 22 19:06 2-dev-role.yaml
-rw-r--r-- 1 Bhavana 197121 301 Jun 22 19:11 3-rolebinding.yaml
-rw-r--r-- 1 Bhavana 197121 220 Jun 22 19:13 4-pod.yaml
```

```
#1-sa-dev.yaml
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dev-sa
  namespace: dev
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl apply -f 1-sa-dev.yaml
Error from server (NotFound): error when creating "1-sa-dev.yaml": namespaces "dev" not found
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl create ns dev
namespace/dev created
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl apply -f 1-sa-dev.yaml
serviceaccount/dev-sa created
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl get sa -n dev
NAME      SECRETS  AGE
default   0         18s
dev-sa    0         13s
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana Work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl api-resources | grep role
clusterrolebindings      rbac.authorization.k8s.io/v1
false                    ClusterRoleBinding
clusterroles              rbac.authorization.k8s.io/v1
false                    ClusterRole
rolebindings              rbac.authorization.k8s.io/v1
true                     RoleBinding
roles                     rbac.authorization.k8s.io/v1
true                     Role
```

```
#2-dev-role.yaml
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
```

```

name: dev-role-podreader
namespace: dev

rules:
- apiGroups: ["" ] # "" indicates the core API group
  resources: ["pods"] #pod belongs empty core API group
  verbs: ["get", "watch", "list"]

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
\$ kubectl apply -f 2-dev-role.yaml
role.rbac.authorization.k8s.io/dev-role-podreader created

```

#3-rolebinding.yaml
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-rolebinding-podreader
  namespace: dev

subjects:
- kind: ServiceAccount
  name: dev-sa
  namespace: dev

roleRef:
  kind: Role
  name: dev-role-podreader
  apiGroup: rbac.authorization.k8s.io

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
\$ kubectl apply -f 3-rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/dev-rolebinding-podreader created

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
\$ kubectl get sa -n dev

NAME	SECRETS	AGE
default	0	4m13s
dev-sa	0	4m8s

```

#4-pod.yaml
---
apiVersion: v1
kind: Pod
metadata:
  name: test-ns-pod-reader
  namespace: dev
spec:
  serviceAccountName: dev-sa #here we are specifying service account
  containers:
  - name: test
    image: bitnami/kubectl:latest
    command: ["sleep", "3600"]

```



```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl apply -f 4-pod.yaml
pod/test-ns-pod-reader created
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl exec -it test-ns-pod-reader -- bash
Error from server (NotFound): pods "test-ns-pod-reader" not found
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl get po
No resources found in default namespace.
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl get po -n dev
NAME                READY    STATUS    RESTARTS   AGE
test-ns-pod-reader  1/1      Running   0           74s
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl exec -it test-ns-pod-reader -n dev -- bash
I have no name! [ / ]$ kubectl get po
NAME                READY    STATUS    RESTARTS   AGE
test-ns-pod-reader  1/1      Running   0           93s
I have no name! [ / ]$ kubectl delete po test-ns-pod-reader
Error from server (Forbidden): pods "test-ns-pod-reader" is forbidden: User "system:serviceaccount:dev:dev-sa" cannot delete resource "pods" in API group "" in the namespace "dev"
I have no name! [ / ]$ exit
exit
command terminated with exit code 1
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl auth can-i get pods \
  --as=system:serviceaccount:dev:dev-sa \
  -n dev
yes
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl auth can-i delete pods --as=system:serviceaccount:dev:dev-sa -n dev
no
```

```
#5-clusterrole.yaml
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: ["" ]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)
$ kubectl apply -f 5-clusterrole.yaml
clusterrole.rbac.authorization.k8s.io/node-reader created
```

```
# 6-clusterrolebinding.yaml
---
```

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-reader-binding

subjects:
- kind: ServiceAccount
  name: dev-sa
  namespace: dev
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl apply -f 6-clusterrolebinding.yaml
clusterrolebinding.rbac.authorization.k8s.io/node-reader-binding created
```

```

ServiceAccount (dev-sa)
|
ClusterRoleBinding
|
ClusterRole (node-reader)
|
Can read Nodes across the cluster

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl get clusterrole node-reader
NAME          CREATED AT
node-reader    2026-06-23T00:47:09Z
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl describe clusterrole node-reader
Name:          node-reader
Labels:        <none>
Annotations:    <none>
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----
  nodes          []                  []              [get list watch]
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl get clusterrolebinding node-reader-binding
NAME                     ROLE                      AGE
node-reader-binding      ClusterRole/node-reader    3m58s
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl describe clusterrolebinding node-reader-binding
Name:          node-reader-binding
Labels:        <none>
Annotations:    <none>
Role:
  Kind: ClusterRole
  Name: node-reader
```


Subjects:

Kind	Name	Namespace
ServiceAccount	dev-sa	dev

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl auth can-i get nodes \
--as=system:serviceaccount:dev:dev-sa
Warning: resource 'nodes' is not namespace scoped
```

yes

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl auth can-i list nodes \
--as=system:serviceaccount:dev:dev-sa
Warning: resource 'nodes' is not namespace scoped
```

yes

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ kubectl auth can-i delete nodes \
--as=system:serviceaccount:dev:dev-sa
Warning: resource 'nodes' is not namespace scoped
```

no

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ eksctl get cluster
NAME          REGION    EKSCtl  CREATED
clusterrbac   us-east-1  True
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/19_June-RBAC-ServiceAccount (main)

```
$ eksctl delete cluster clusterrbac
```